DREW

Lecture 35: Struct Datatype in C

CSCI 260 Computer Systems & Architecture
Fall 2024

Andy Siyuan Feng

11/25/2024

Data Structure

- Earlier, looked at arrays.
- An array is *a collection of memory addresses and associated contents organized* in a linear fashion; i.e., a continuous chunk those.
    - There's a certain way to access those locations and data. [**Index**]
- To generalize that, any *Data Structure* is such a collection organized in some certain ways in its reserved memory chunk.
    - There's a certain way to access those locations and data.

A User-Defined Data Structure

- Recall an array can only organize contents of a single datatype.
  - int arr[20] double arr[20] char arr[20]

- In C, we can define a *struct* that can organize contents of different datatypes.

- **Ex.**: How do we keep an entry of a student record in C?
  - A char[] for name, an int for ID#, a char for graduate class or not, etc.
  - Need all those for every student.

  - How about using only 1 variable?

A User-Defined Data Structure

- **Ex.**: How do we keep an entry of a student record in C?

- By using *struct*, here's how we can do it:
  - **Idea**: Create a "container" (object??) that contains all the attributes.

    For each student, instantiate a new such container and initialize all the attributes with specific values.

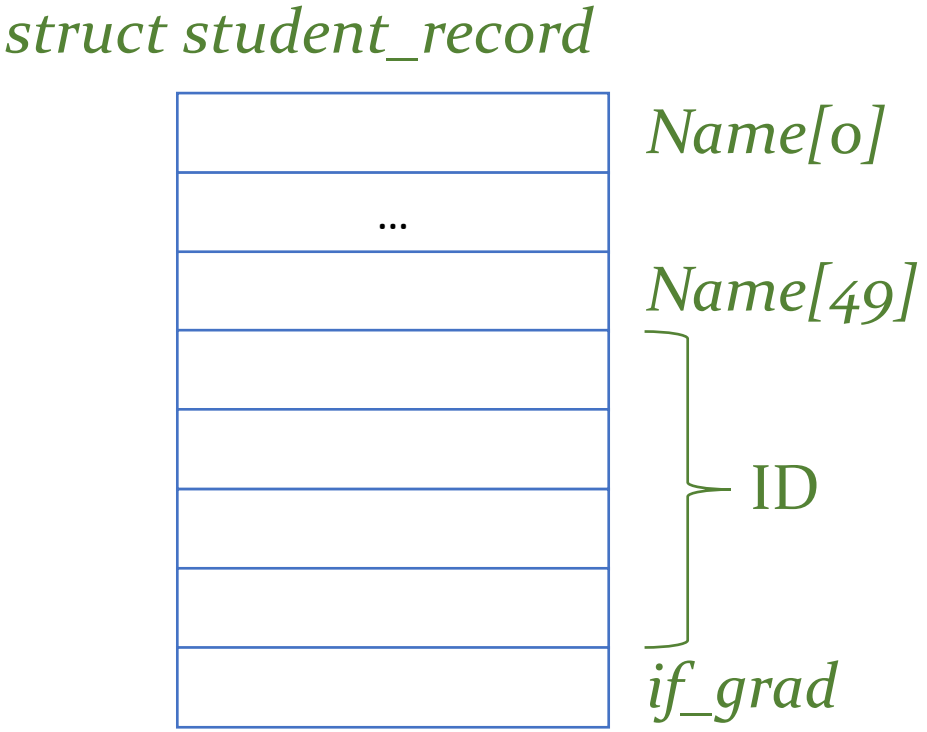
  - **Implementation**:

```
struct student_record {
        char* Name;
        int ID;
        char if_grad;
};
```

DREW

Define struct

- **Implementation**: *struct student_record {*
  char Name;*
  int ID;
  char if_grad;
  };



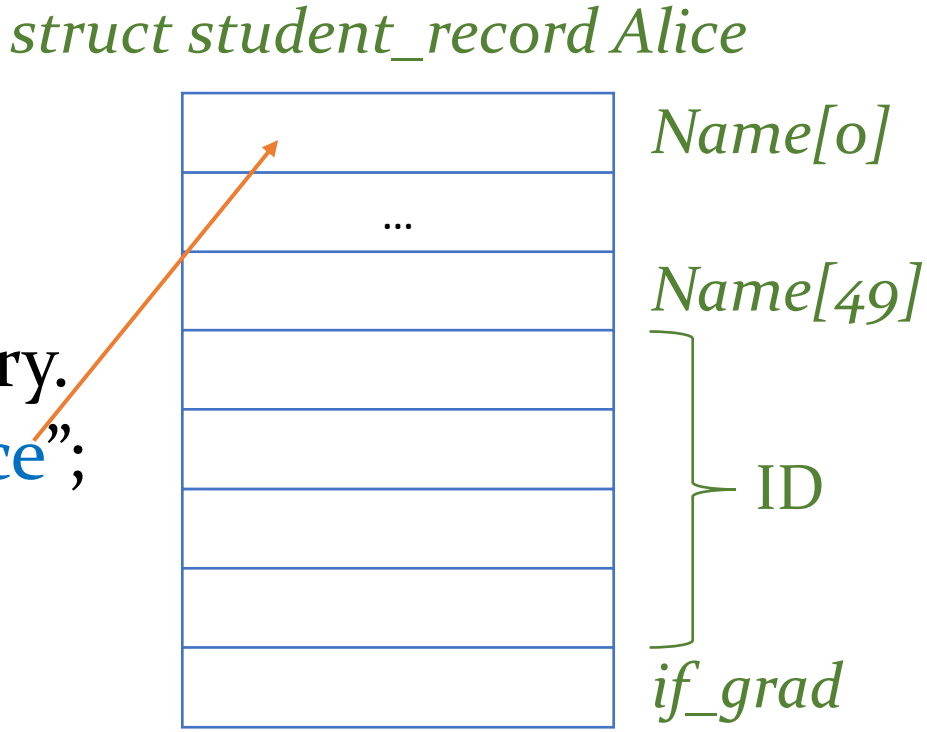
- This declaration specifies how the addresses and contents in this data structure are to be organized in the memory. And how big is the structure.
  - 50 chars, each 1 byte; 1 int, 4 bytes; 1 char, 1 byte.
  - Everything in a continuous chunk.

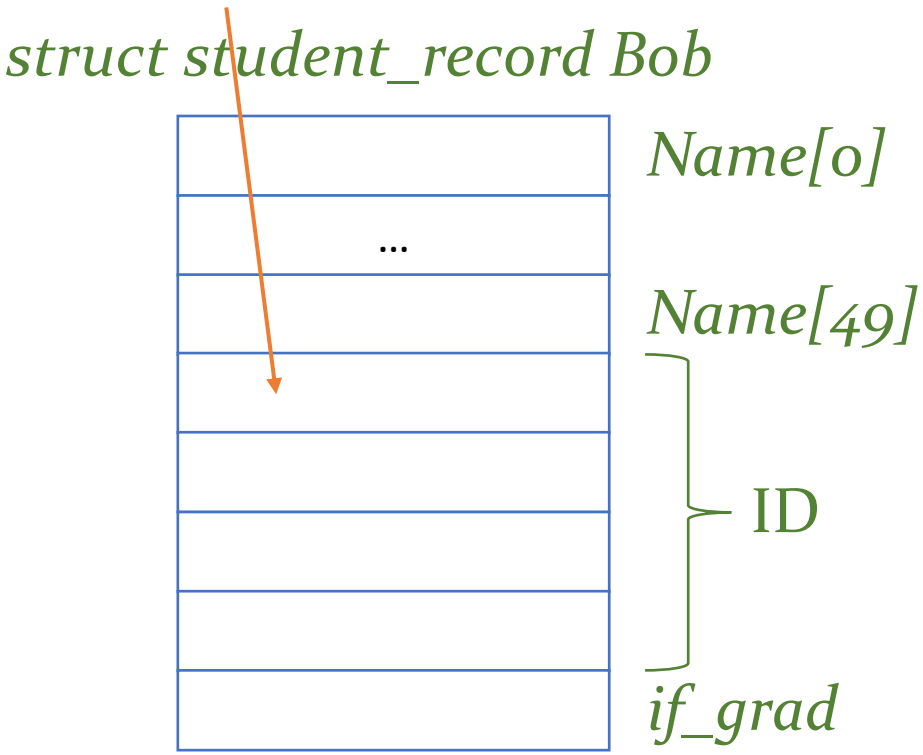
Define struct

Say, we instantiated 2 instances, *Alice* and *Bob*.

- Then, two such organizations in the memory.
- Can now set value of *Name* for *Alice* as "Alice";

set value of *ID* for *Bob* as 12345.





Implement struct

- To instantiate an instance of the struct, we simply treat

struct student_record as a datatype; i.e., to instantiate *John*:

struct student_record John; .

- Note here *struct student_record* is the datatype, where the keyword *struct* must always be included, otherwise *student_record* will be treated as a var name.
- Here's a **shortcut**: Use the *typedef* function to re-name a datatype something else.
- Here's what we can do, *typedef struct student_record SR;* .
- Then, to instantiate *John*: *SR John;* .

DREW

Implement struct

- Here's how to combine *typedef* and *struct*:

```
typedef struct student_record {
        char* Name;
        int ID;
        char if_grad;
} SR;
```

In *main()*:

```
SR John, Alice, Bob;        // Instantiate individual vars.
SR CSCI260[25];             /* Instantiate an array of SRs, where
                               each index location contains an SR. */
```

Implement struct

- Now, to access the attribute values of a struct instance, we use the "." notion (recall an object!!).

- **Ex.**:

```
SR Paul;
 Paul.Name = "Paul McCartney";
 Paul.ID = 987654;
 Paul.if_grad = 'F';

printf("Name: %s\tID: %d\tIf graduating: %c\n",
                                    Paul.Name, Paul.ID, Paul.if_grad);
```

Implement struct

- Now, to access the attribute values of a struct instance, we use the "." notion (recall an object!!).

- **Ex.**:

```
SR CSCI260[25];
CSCI260[17].Name = "Bob";
…

printf("Name: %s\tID: %d\tIf graduating: %c\n",
                                    CSCI360[17].Name, …);
```

Implement struct

- What if we have a bunch of SRs as an array (*SR CSCI260[25];*), and we want to extract one SR, not just its attributes, but as a whole SR?

- **Ans**: Can define a pointer that is pointing towards that specific SR in the SR array; i.e., an SR pointer.
- **Ex.**:

```
SR CSCI260[25]; … …
SR* John = &CSCI260[21];

printf("Name: %s"\n, (*John).Name);        // MUST have ()!
(*John).if_grad = 'T';      // updating record
John->if_grad = 'T';        // is exactly the same as above!!
```

Passing struct

- Speaking of pointers, how about passing *struct* as arg.?
- Turns out, unlike arrays, struct are passed by values.
- Thus, need to pass the address of a struct instead using pointer!

- **Ex.**:

```
#include <string.h>

int same_name (SR* s1, SR* s2) {
        if (strcmp(s1->name, s2->name) == 0)        // 0 if same
                return 1;
        else return 0;
}
```

DREW

Dynamic Allocation

- Recall when we were talking about arrays, we can dynamically declare an array using *malloc()* so we can create enough, but not too many wasted memory locations.
- Similarly here for *struct*:

```
SR* John = (SR*) malloc(sizeof(SR));
if (John == NULL) return 1;      // Check if successful

SR* DSEM001 = (SR*) malloc(sizeof(SR)*varLen);
DSEM001[3].Name = "Ringo";
...       free(John); free(DSEM001);
```

Dynamic Allocation

- Why do we need this?

- A specific data structure, *Linked List*, will need to use *struct* to create "*nodes*", where one of the attribute of a *node* is pointing towards another *node*.
- So, struct pointer! Will need *malloc()* to instantiate it.